

INSTITUT NATIONAL DES SCIENCES
APPLIQUÉES DE ROUEN

INSA DE ROUEN



GM3, PROJET MIPP

EmoSense

Auteurs :

Manh Hung NGUYEN

mnguy00@insa-rouen.fr

Tan Minh Duy NGO

tn00@insa-rouen.fr

Enseignant :

Aomar OSMANI

aomar.osmani@insa-rouen.fr

7 Février 2026

Résumé

Ce projet s'inscrit dans le cadre de l'unité d'enseignement de Programmation Orientée Objet (POO) et porte sur le développement d'une application d'analyse de sentiments en langage Java, intitulée EmoSense.

L'objectif principal est de concevoir une architecture logicielle modulaire capable d'évaluer la charge émotionnelle d'un texte (joie, colère, tristesse, etc.) en confrontant deux approches techniques distinctes. La première repose sur un algorithme interne "Rule-Based", utilisant des lexiques pondérés et des règles grammaticales heuristiques. La seconde simule l'appel à une Intelligence Artificielle externe (Google Gemini API) pour obtenir une analyse contextuelle avancée.

La réalisation de ce projet nous a permis de mettre en œuvre des concepts avancés de conception logicielle, notamment l'utilisation d'interfaces et du patron de conception Strategy pour assurer l'extensibilité du système. Le rapport met en lumière les performances de l'approche algorithmique pour le traitement rapide de textes structurés, tout en soulignant ses limites face aux subtilités linguistiques (ironie, argot) mieux gérées par les modèles d'IA modernes.

Keywords: *Analyse de sentiments, NLP, Java, Programmation Orientée Objet*

Table des matières

1	Introduction	2
2	Cahier des charges et Expression des besoins	3
3	Architecture et Choix UML	7
3.1	Organisation en Packages	7
3.2	Diagramme de Classe	8
3.3	Diagramme de Séquence	9
4	Méthodologie d'analyse	12
5	Résultat et Limites	16
5.1	Analyse des résultats	16
5.2	Limites et Critique de l'algorithme	17
6	Utilisation des IAs	19
7	Conclusion	21

Chapitre 1

Introduction

De nos jours, le traitement du langage naturel (NLP - Natural Language Processing) est devenu un domaine incontournable de l'informatique, notamment pour comprendre les opinions et les émotions exprimées sur les plateformes numériques. C'est dans ce contexte que s'inscrit notre projet MIPP de programmation en Java, intitulé EmoSense.

L'objectif principal de ce projet est de concevoir une application console capable d'analyser la charge émotionnelle d'un texte donné. Au-delà de la simple polarité (positive, négative ou neutre), le système vise à identifier des nuances plus fines (comme la joie, la colère ou la tristesse) et à attribuer un score de sentiment précis. Ce travail a pour but de confronter deux approches techniques distinctes : Une approche algorithmique basée sur des règles ("Rule-Based"), utilisant un lexique pondéré et une analyse grammaticale et une approche comparative via une API externe ("External API"), simulant l'appel à une intelligence artificielle.

Ce projet a été l'occasion idéale de mettre en pratique les concepts fondamentaux de la Programmation Orientée Objet, tels que l'encapsulation, le polymorphisme et l'héritage. Nous avons porté une attention particulière à l'architecture logicielle, notamment par l'utilisation de Design Patterns afin de rendre le code modulaire, maintenable et extensible.

Ce rapport détaille l'ensemble de notre démarche, depuis l'analyse des besoins et la modélisation UML jusqu'à l'implémentation technique et la justification de nos choix de conception.

Chapitre 2

Cahier des charges et Expression des besoins

Dans un contexte où les avis utilisateurs et les commentaires sur les réseaux sociaux sont omniprésents, il devient essentiel de pouvoir classer automatiquement ces textes selon leur charge émotionnelle.

La problématique centrale de ce projet est la suivante : Comment concevoir une architecture logicielle modulaire permettant de confronter une analyse algorithmique interne avec une analyse issue d’une intelligence artificielle externe ?

Le projet se limite aux contraintes techniques et fonctionnelles suivantes :

- **Langage** : Java.
- **Interface** : Application Console pour simplifier les interactions.
- **Données** : Utilisation de fichiers au format texte (`.txt`) pour le lexique émotionnel, de fichier JSON (`.json`) pour les rapports d’analyse et de fichier image (`.png`) pour les visualisations graphiques.
- **Architecture** : Utilisation obligatoire de la Programmation Orientée Objet (POO), incluant l’usage d’interfaces et de modèles de conception.

Le système doit répondre aux exigences fonctionnelles suivantes, classées par ordre de priorité :

- **RF1 - Analyse Lexicale (Rule-Based)** : Le système doit être capable de charger deux dictionnaire distincts (`emotion_lexicon` et `marker_lexicon`) pour identifier non seulement les mots porteurs d’émotion (ex : "happy", "sad"), mais aussi les modificateurs grammaticaux (négation, intensification). L’algorithme doit calculer un score précis sur une échelle de 0 à 20 et identifier l’émotion dominante (Joie, Colère, Tristesse, Peur, etc.).

- **RF2 - Simulation d'API Externe** : Afin de valider la pertinence de l'algorithme interne, le système doit simuler un appel à une API tierce. Cette simulation permettra de récupérer un "score de référence" pour comparaison.
- **RF3 - Comparaison et Visualisation** : L'application doit afficher les résultats des deux méthodes côte à côte. De plus, elle doit générer un diagramme circulaire (Pie Chart) illustrant la distribution des émotions détectées (ex : 60% Joie, 40% Surprise) pour une meilleure lisibilité.
- **RF4 - Exportation des Données** : L'utilisateur doit avoir la possibilité de sauvegarder le résultat de l'analyse dans un fichier au format JSON structuré.
- **RF5 - Gestion des Erreurs** : Si l'utilisateur estime que l'analyse est incorrecte, il doit pouvoir le signaler. Le système enregistrera alors l'entrée problématique dans un fichier de log (`errors.log`).

Outre les fonctionnalités, le système doit respecter des critères de qualité logicielle :

- **RNF1 - Extensibilité (Open/Closed Principale)** : L'architecture doit permettre l'ajout futur de nouveaux algorithmes d'analyse (Machine Learning) sans modifier le code existant. L'utilisation de l'interface `IEmotionAnalyzer` est donc impérative.
- **RNF2 - Maintenabilité** : Le code doit être organisé en *packages* logiques (`core`, `model`, `io`, `app`) pour séparer les responsabilités.
- **RNF3 - Robustesse** : Le programme ne doit pas planter si le fichier dictionnaire est manquant ou si l'utilisateur saisit des données invalides (gestion des exceptions).

Afin de visualiser les interactions entre l'utilisateur et le système, nous avons établi un diagramme de cas d'utilisation.

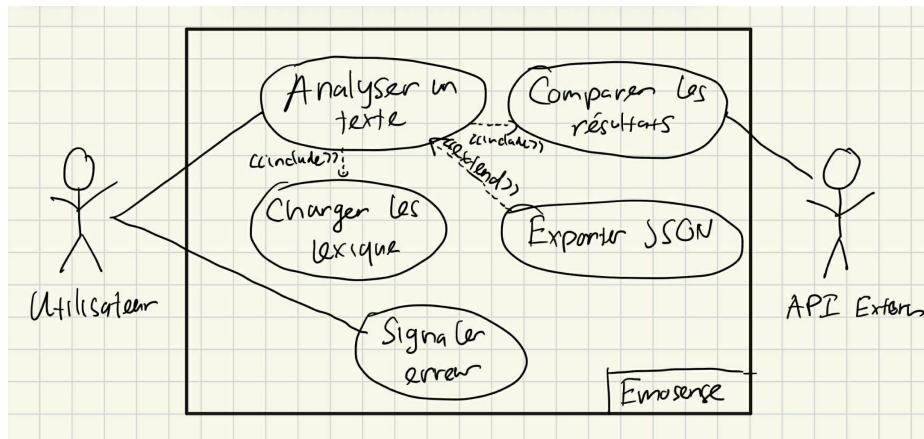


FIGURE 2.1 – Diagramme de cas

Description Textuelle des Cas d'Utilisation

Identification :	
Nom du cas	Analyser un texte et comparer les scores
But	Permettre à l'utilisateur d'obtenir une double évaluation émotionnelle d'une phrase (Algorithme interne vs API) et d'en visualiser les détails.
Acteurs	Principal : Utilisateur Secondaire : API Externe (Simulée)
Date	15 février 2026
Responsables	NGUYEN Manh Hung & NGO Tan Minh Duy
Version	2.0

TABLE 2.1 – Identification du cas d'utilisation "Analyser un texte"

Les séquencements :	
Pré-conditions	— L'application est lancée. — Le fichiers de lexique a été chargé avec succès en mémoire.
Enchaînements	1. Le système affiche le menu principal et invite l'utilisateur à choisir une action. 2. L'utilisateur sélectionne l'option "Analyser" et saisit une phrase (ex : "I am not only happy but also excited!"). 3. Le système découpe le texte en Tokens et lance l'analyse via le moteur de règles (RuleBasedAnalyzer). 4. Le système interroge l'API Externe pour obtenir une seconde évaluation. 5. Le système affiche les deux résultats sur la console, ainsi que l'explication détaillée du calcul (mots détectés, bonus appliqués). 6. Le système génère automatiquement un graphique (Pie Chart) de la distribution émotionnelle. 7. Le système propose de sauvegarder les résultats en JSON.
Post-conditions	— Les scores sont affichés à l'écran. — Les fichiers de sortie sont créés si demandé. - Le système revient au menu principal.
Scénarios Alternatifs	Erreur de Saisie : Si l'utilisateur entre une chaîne vide, le système affiche un avertissement et redemande une saisie. Panne API : Si l'API ne répond pas, le système utilise un résultat "Neutre" par défaut et signale l'indisponibilité, sans bloquer l'analyse locale.

TABLE 2.2 – Séquencements du cas d'utilisation "Analyser un texte"

Chapitre 3

Architecture et Choix UML

3.1 Organisation en Packages

Afin de respecter le principe de Séparation des Responsabilités, nous avons divisé le code source en plusieurs packages distincts, chacun ayant un rôle précis :

- **app** : Contient le point d'entrée de l'application (**EmoSenseApp**). C'est la couche de présentation qui gère l'interaction avec l'utilisateur (Menu, Saisie, Affichage).
- **interfaces** : Définit les contrats communs, notamment **IEmotionAnalyzer**, permettant d'abstraire les différentes méthodes d'analyse.
- **core** : Contient le cœur logique de l'algorithme "Rule-Based". Il est subdivisé en :
 - **core.lexicon** : Gestion des dictionnaires et recherche de mots.
 - **core.rule** : Moteur de règles et logique de calcul.
 - **core.text** : Prétraitement et tokenisation du texte.
- **api** : Gère la communication HTTP avec le service d'intelligence artificielle externe.
- Contient les objets de données (POJO/Records) tels que **Tokens**, **AnalysisResult**, **Contribution**. Ces classes servent de pivot pour transférer les données entre les couches.
- **io** : Gère les entrées/sorties, notamment l'exportation des résultats en JSON et la génération des graphiques PNG.

3.2 Diagramme de Classe

Le diagramme de classes est la représentation centrale de notre architecture orientée objet. Il définit la structure statique du système EmoSense, en détaillant les attributs, les méthodes et les relations entre les différents modules.

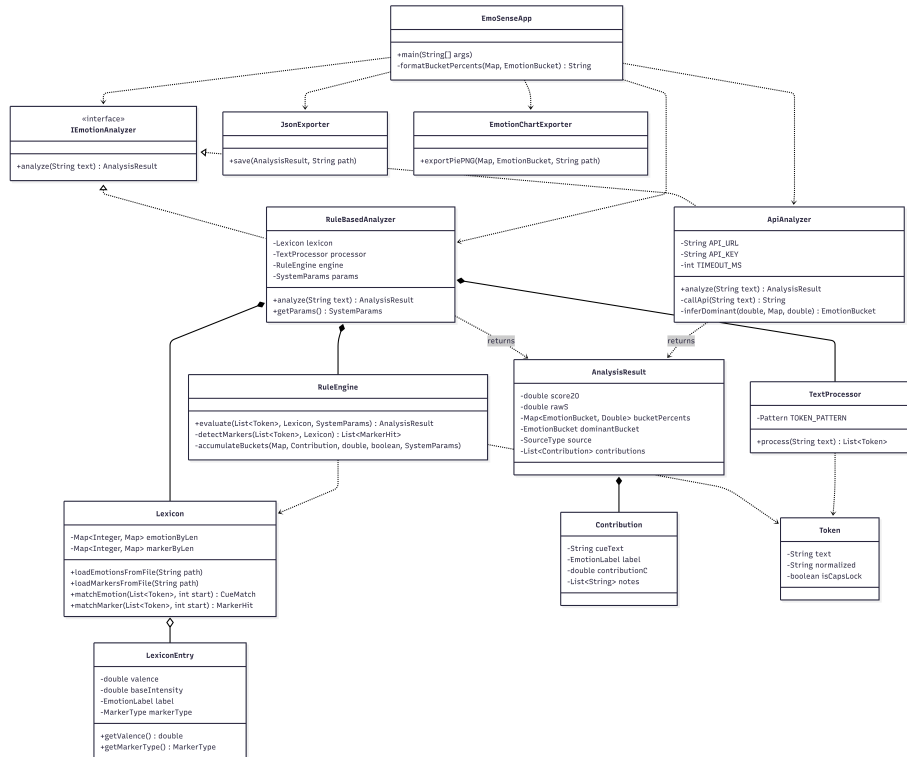


FIGURE 3.1 – Diagramme de Classe

Au sommet de la hiérarchie se trouve l'interface **IEmotionAnalyzer**. Elle définit le contrat unique que tout analyseur doit respecter : la méthode **analyze(String text)** renvoyant un **AnalysisResult**. Cette abstraction est la mise en œuvre du patron de conception **Strategy**. Elle permet à l'application principale de manipuler indifféremment une analyse algorithmique (**RuleBasedAnalyzer**) ou une analyse externe (**ApiAnalyzer**) grâce au polymorphisme. Cela rend le système ouvert aux extensions futures.

La classe **RuleBasedAnalyzer** ne fonctionne pas seule. Elle agit comme un Contrôleur qui orchestre plusieurs composants spécialisés via des relations de Composition :

- **TextProcessor** : Chargé exclusivement du nettoyage et du découpage du texte (Tokenization). Il transforme une **String** brute en une liste d'objets **Token**.
- **Lexicon** : C'est la base de données en mémoire. Il stocke les dictionnaires chargés depuis les fichiers **.txt** et fournit des méthodes de recherche rapide.
- **RuleEngine** : C'est le cerveau mathématique. Il reçoit les tokens et le lexique, applique les règles de pondération (négation, intensification, majuscules) et retourne le résultat final.

Contrairement à une approche procédurale qui utiliserait des variables primitives, nous avons modélisé les données via des objets riches et immuables :

- **AnalysisResult** : Encapsule toutes les informations de sortie (score sur 20, distribution des émotions, liste des contributions).
- **Contribution** : Permet la traçabilité. Chaque mot ayant influencé le score est stocké dans cet objet, accompagné de notes explicatives (ex : "Boosté par CAPS LOCK").
- **EmotionBucket** : L'utilisation d'une énumération permet de catégoriser finement les émotions (ex : **JOY_POS**, **ANGER_NEG**).

3.3 Diagramme de Séquence

Si le diagramme de classes montre ce que sont les objets, le diagramme de séquence montre ce qu'ils font et comment ils interagissent dans le temps.

Le diagramme suivant détaille le scénario nominal : L'utilisateur demande l'analyse d'une phrase via l'algorithme Rule-Based et exporte les résultats.

3. **Calcul et Construction du Résultat** : Une fois les mots identifiés :
 - Le moteur effectue des calculs internes pour appliquer les pondérations.
 - Il instancie un objet `AnalysisResult` immuable contenant toutes les statistiques.
 - Cet objet remonte la chaîne d'appel : `RuleEngine` $\rightarrow$ `RuleBasedAnalyzer` $\rightarrow$ `EmosenseApp`.
4. **Restitution et Exportation** : Enfin, `EmosenseApp` reprend la main pour la gestion des Entrées/Sorties. Le diagramme montre un bloc par illustrant deux actions indépendantes :
 - Génération Graphique : Appel à `motionChartExporter.exportPiePNG()` pour créer l'image du diagramme circulaire.
 - Sauvegarde JSON : Appel à `JsonExporter.save()` pour sérialiser l'objet résultat en fichier texte.

Cette séparation finale confirme que la logique métier (calculer le sentiment) est totalement découplée de la logique de présentation (afficher ou sauvegarder le résultat).

Chapitre 4

Méthodologie d'analyse

Contrairement aux approches de Machine Learning, notre approche Rule-Based est déterministe et explicable. Elle repose sur une combinaison d'analyse lexicale (dictionnaires) et d'analyse syntaxique (règles grammaticales heuristiques). Ce chapitre détaille les mécanismes mathématiques implémentés dans le moteur **RuleEngine**.

S

structure des Données Lexicales

Le système ne se contente pas de compter des mots positifs ou négatifs. Il s'appuie sur deux bases de connaissances distinctes chargées au démarrage.

1. **Le Lexique Émotionnel** (`emotion_lexicon_v1.txt`) : Ce fichier associe des mots ou des expressions à une empreinte émotionnelle complexe. Chaque entrée est définie par quatre vecteurs :
 - Le Label : La catégorie primaire (JOY, SADNESS, ANGER, FEAR, DISGUST, SURPRISE).
 - La Valence (v) : Une valeur comprise entre -1.0 (négatif) et $+1.0$ (positif).
 - L'intensité de base (a) : Une valeur comprise entre 0.0 et 2.0 , indiquant la force intrinsèque du mot.

Exemple : Le mot "furious" possède une valence négative forte $v = -0.95$ et une intensité élevée $a = 1.85$, tandis que "annoyed" est plus faible $v = -0.55$, $a = 0.80$.

2. **Le Lexique des Marqueurs** (`marker_lexicon_v1.txt`) : Ce fichier contient les modificateurs grammaticaux qui altèrent le sens des mots voisins. Ils sont classés par types :
- INT (Intensifier) : Augmente l'impact (ex : *very, extremely*).
 - DIM (Diminisher) : Réduit l'impact (ex : *slightly, a bit*).
 - NEG (Negation) : Inverse la polarité (ex : *not, never*).
 - CON (Contrast) : Bascule le contexte (ex : *but, however*).
 - ESC (Escalation) : Renchérit sur une idée (ex : *not only... but also*).

Prétraitement et Tokenisation

Avant l'analyse, le texte brut passe par le `TextProcessor`. Nous avons fait le choix technique de ne pas supprimer la ponctuation expressive (ex : `!`, `?`) ni les émojis, car ils sont porteurs de sens. De plus, chaque mot est stocké dans un objet `Token` qui retient si le mot était écrit en MAJUSCULES, une information cruciale pour détecter la colère ou l'enthousiasme .

Algorithme de Correspondance (N-Grams Matching)

Une difficulté majeure du NLP est de reconnaître les expressions composées (ex : "fed up" ou "out of blood"). Notre algorithme applique une stratégie de correspondance par le plus long préfixe (*Longest Matching Strategy*). Pour chaque position dans le texte, le système vérifie d'abord si une séquence de 3 mots correspond à une entrée du lexique. Si non, il essaie avec 2 mots, puis 1 mot. Cela évite, par exemple, que "at ease" (JOY) ne soit détecté à tort comme juste "at" (neutre).

Règles Heuristiques et Pondération

Une fois un mot émotionnel (*Cue*) identifié, le `RuleEngine` calcule son score final (S_{mot}) en inspectant son voisinage.

La formule générale appliquée pour chaque mot est :

$$S_{mot} = v_{final} * a_{final} * w_{segment}$$

Où les variables sont modifiées dynamiquement par les règles suivantes :

1. **La Règle de Négation** : Le système regarde les 4 mots précédents. Si un marqueur NEG est trouvé :
 - La valence est inversée et atténuée : $v_{final} = v_{base} * -0.75$.
 - L'intensité est réduite : $a_{final} = a_{base} * 0.8$.
 - *Justification* : Dire "*I am not happy*" est négatif, mais moins fort que "*I am sad*".
2. **La Règle d'Itensification et de Majuscules**
 - Adverbes : Si un INT (ex : *very*) précède le mot, l'intensité a est multipliée par le facteur du marqueur (ex *1.35).
 - Caps Lock : Si le mot est écrit en majuscules (ex : "*HAPPY*"), l'intensité est boostée de 15%.
 - Ponctuation : La présence de !! ou ?! juste après le mot ajoute un multiplicateur supplémentaire.
3. **La Règle de Contraste (Segmentation)** : Les conjonctions comme "*but*" ou "*however*" divisent la phrase en deux segments.
 - Le segment avant le contraste voit son poids réduit ($w = 0.7$).
 - Le segment après le contraste voit son poids augmenté ($w = 1.1$).
 - *Example* : Dans "*The film is long but good*", l'avis "*good*" prévaut sur "*long*".

Normalisation et Calcul du Score Final

Le score brut du texte est la somme algébrique de toutes les contributions individuelles :

$$RawS = \sum S_{mot}$$

Pour convertir ce score brut (qui peut théoriquement aller de $-\infty$ à $+\infty$ en une note scolaire comprise entre 0 et 20 nous utilisons une fonction de tangente hyperbolique (*tanh*). Cette méthode assure une distribution fluide et évite les effets de seuil brutaux :

$$Score_{20} = 10 + 10 \times \tanh \left(1.5 \times \frac{RawS}{S_{max}} + 0.1 \right)$$

Où S_{max} est le score maximal théorique possible pour la longueur de la phrase donnée..

Enfin, le système ne se contente pas d'une moyenne. Chaque contribution est distribuée dans des "seaux" (*Buckets*).

- Si un mot a une valence forte, son score va dans le bucket polarisé (ex : `JOY_POS`).
- Si un mot a une valence faible ou est sous l'effet d'une négation, une partie de son score est redirigée vers un bucket "Neutre" (ex : `JOY_NEU`).

Ceci permet de générer le diagramme circulaire final qui montre la complexité des émotions (ex : "80% Colère, 20% Tristesse").

Chapitre 5

Résultat et Limites

Afin de valider la robustesse de notre application EmoSense, nous avons soumis le système à une batterie de tests rigoureux, allant de phrases grammaticalement complexes à des cas limites. Ce chapitre présente les résultats obtenus et analyse en toute transparence les limitations inhérentes à notre approche algorithmique.

Nous avons réussi l'implémentation technique complète de la classe **ApiAnalyzer**. Le code est capable de construire une requête HTTP conforme, d'envoyer un Prompt système précis et de parser la réponse JSON complexe de Google Gemini. Cependant, lors de la phase de validation finale, nous avons rencontré une contrainte externe liée aux quotas d'utilisation du modèle d'Intelligence Artificielle. Le modèle gratuit (Free Tier) étant saturé ou restreint géographiquement, l'API renvoie une erreur ou ne traite pas la demande. Par mesure de sécurité et pour éviter le plantage de l'application, nous avons implémenté une méthode **fallbackNeutral()** qui renvoie un résultat par défaut (Score : 10/20, Neutre) en cas d'indisponibilité du service.

Par conséquent, la comparaison directe en temps réel entre l'algorithme manuel et l'IA n'a pas pu être effectuée sur l'ensemble des tests, bien que l'architecture logicielle soit prête pour le faire.

5.1 Analyse des résultats

Notre moteur de règles a démontré une excellente capacité à gérer les nuances de la langue anglaise grâce aux mécanismes de pondération implémentés. Voici trois exemples tirés de nos logs de test **result.txt**.

1. **Gestion de l'intensification et de l'escalade.** Le système détecte correctement les structures cumulatives.
 - Test : *"I am not only happy, but also thrilled about this opportunity !"*
 - Résultat : Score de 17.89/20 (Très positif).
 - Observation : L'algorithme a identifié les marqueurs **ESC** (Escalation). Le mpt *"happy"* a été amplifié par *"not only"* et *"thrilled"* par *"but also"*. L'accumulation de ces bonus permet de dépasser le score standard d'un mot seul.
2. **Gestion du contraste("But") :** Le système est capable de pondérer différemment les parties d'une phrase.
 - Test : *"The movie was long and boring, but the ending was fantastic."*
 - Résultat : Score de 11.75/20 (Positif).
 - Observation : Bien que la première partie soit négative (*"boring"*), le marqueur **but** a permis de réduire son impact (poids $w = 0.7$) et d'augmenter l'importance de la fin de phrase (*"fantastic"*, poids $w = 1.1$). Le score final reflète correctement l'opinion globale.
3. **Gestion de la négation :**
 - Test : *"I am not happy with this result."*
 - Résultat : Score de 7.88/20 (Négatif).
 - Observation : Le système n'a pas été trompé par la présence du mot *"happy"*. Il a détecté le marqueur **NEG** *"not"* dans la fenêtre contextuelle, inversant la valence positive en négative.

5.2 Limites et Critique de l'algorithme

Malgré ces succès, nos tests de robustesse ont révélé plusieurs faiblesses structurelles inhérentes à une approche basée sur des règles.

Dépendance stricte au Lexique

Le système est totalement dépendant de son dictionnaire (`emotion_lexicon_v1.txt`). Si un mot est mal orthographié, collé à un autre, ou dans une autre langue, il est ignoré.

- Exemple : L'entrée *"Iamveryhappybutalsosad"* produit un score Neutre (11.00/20) car le tokenizer ne peut pas séparer les mots.
- Exemple : Des caractères spéciaux ou du "bruit" (ex : `$$$ %%`) ne déclenchent aucune émotion. Contrairement à une IA qui pourrait inférer le sens, notre système suit une logique stricte : pas de mot connu égale pas d'émotion.

Limites de la "Fenêtre" (Window Logic)

L'association entre un marqueur (ex : "very", "not") et un mot émotionnel repose sur une proximité physique (fenêtre de quelques mots). Dans des phrases longues comme "not only happy... but also excited", le lien grammatical est parfois mal interprété. Le mot "excited" est techniquement boosté par le marqueur le plus proche ("also"), mais le système ne comprend pas la structure globale de la phrase comme un tout cohérent. Si la phrase est trop complexe, l'influence des marqueurs peut être attribuée à tort ou perdue.

Gestion simplifiée du Contraste

Notre gestion du contraste est binaire. Le système détecte un pivot (ex : "but", "however") et divise la phrase en deux : "avant" et "après". Cependant, Cette approche échoue sur les phrases à tiroirs ou avec des contrastes multiples (ex : "I like him, but he went, however i still good"). Le système ne peut gérer qu'un seul niveau de bascule émotionnelle principal.

Nature multiplicative des modificateurs (Caps Lock & Punctuation)

Les majuscules et la ponctuation agissent uniquement comme des multiplicateurs (boosters). Par conséquent, Une phrase comme "!!!!" ou "I AM HERE" renvoie un score neutre. Puisque ces mots ne portent pas d'émotion intrinsèque (valeur 0), les multiplier par 1.5 (pour les majuscules ou les points d'exclamation) donne toujours 0. Il faut obligatoirement un mot clé émotionnel pour que ces artifices aient un effet

Troncature des résultats

Pour des raisons de lisibilité dans la console, nous avons limité l'affichage des détails aux 10 contributions les plus fortes. Si une phrase contient plus de 10 mots émotionnels, les nuances mineures ne sont pas affichées à l'utilisateur, bien qu'elles soient prises en compte dans le calcul du score brut (Raw Score).

Chapitre 6

Utilisation des IAs

Dans le cadre de ce projet, nous avons adopté une approche moderne du développement logiciel en intégrant des outils d’Intelligence Artificielle Générative (notamment Google Gemini et ChatGPT) comme assistants de production. Loin de remplacer le travail de conception, ces outils ont agi comme des accélérateurs de développement, nous permettant de nous concentrer sur la logique métier complexe et l’architecture.

Assistance au Développement et au Débogage

L’apport le plus significatif de l’IA a concerné l’écriture et l’optimisation du code Java, en particulier pour les tâches répétitives ou techniquement verbeuses.

- Gestion de l’API Externe : L’intégration de l’API Google Gemini nécessitait la construction manuelle de requêtes HTTP et le parsing de réponses JSON complexes. L’IA nous a aidés à générer les expressions régulières pour extraire proprement les champs, `score20` et `bucketPercent` sans utiliser de bibliothèques externes lourdes, allégeant ainsi le projet.
- Correction d’erreurs : Face aux erreurs de compilation ou d’exécution, l’IA a servi d’outil de diagnostic rapide, proposant des correctifs immédiats.
- Refactoring et Clean Code : L’IA a suggéré l’utilisation du Pattern Strategy pour l’interface `IEmotionAnalyzer`, rendant le code plus modulaire et extensible.

Génération de Données et de Scénarios de Test (Data)

La création manuelle de dictionnaires et de jeux de tests est une tâche fastidieuse et sujette à l'erreur humaine. Nous avons automatisé ce processus : Enrichissement des Lexiques : Les fichiers dictionnaires ont été générés en partie par l'IA. Nous lui avons demandé de fournir une liste de 100+ adjectifs anglais avec leur valence (-1.0 à +1.0) et leur intensité, ce qui a permis d'obtenir une base de connaissances riche et variée en quelques minutes.

Assistance à la Conception Architecturale (UML)

La modélisation visuelle du système a été facilitée par la génération de code pour des outils de diagrammes. Plutôt que de dessiner les diagrammes à la main (ce qui est difficile à modifier), nous avons décrit la structure de nos classes à l'IA, qui a généré le code source correspondant. Cela nous a permis d'obtenir des Diagrammes de Classes et de Séquence professionnels, cohérents avec le code Java, et facilement modifiables.

Rédaction et Synthèse

Enfin, l'IA a été utilisée comme un outil d'aide à la rédaction pour ce rapport.

- Structuration des idées : L'IA a proposé un plan détaillé pour les chapitres techniques (Architecture, Méthodologie), garantissant que tous les points clés (comme le Strategy Pattern ou la logique de Windowing) soient abordés.
- Amélioration linguistique : Étant dans un contexte universitaire, nous avons utilisé l'IA pour reformuler certaines explications techniques afin d'atteindre un niveau de langue soutenu, précis et sans fautes d'orthographe.

Il est important de noter que l'IA n'a pas travaillé en autonomie. Chaque ligne de code générée a été relue, compilée et testée par nous-mêmes. Chaque phrase du rapport a été validée pour s'assurer qu'elle reflétait la réalité du projet.

Chapitre 7

Conclusion

Le projet EmoSense avait pour ambition initiale d'explorer les mécanismes du Traitement Automatique du Langage Naturel (NLP) en confrontant deux paradigmes opposés : l'approche algorithmique traditionnelle et l'intelligence artificielle moderne. Au terme de ce développement, nous avons réussi à produire une application fonctionnelle, modulaire et robuste, capable d'analyser le sentiment d'un texte avec une précision notable.

Les résultats obtenus mettent en lumière une dualité intéressante entre contrôle et intelligence. D'un côté, notre moteur interne Rule-Based s'est révélé extrêmement performant en termes de rapidité d'exécution et de transparence. Il offre une explicabilité totale du score, permettant de tracer mathématiquement chaque variation émotionnelle (négation, intensification, contraste).

Cependant, ce projet n'est pas exempt de limitations. La dépendance stricte de l'algorithme interne envers ses lexiques statiques (.txt). Par ailleurs, l'utilisation de l'API externe nous a confrontés aux réalités économiques du développement logiciel : bien que techniquement fonctionnelle, l'analyse par IA a été freinée par les quotas d'utilisation (Rate Limiting) imposés par le fournisseur.

Si le temps et les ressources nous l'avaient permis, nous aurions souhaité pousser l'analyse au-delà des règles statiques en explorant majeurs d'amélioration via l'Intelligence Artificielle, notamment pour la détection du Sarcasme par Machine Learning, nous envisagerions l'entraînement d'un modèle de Machine Learning supervisé (type SVM ou Réseau de Neurones) sur un corpus de textes annotés (tweets sarcastiques). Ce module viendrait compléter le moteur de règles pour "flagger" les phrases où le sens littéral contredit le sens figuré.